

Monitoring and troubleshooting

CloudWatch, CloudTrail, log analysis at scale, and diagnosing a pipeline that has gone wrong.

Where logs actually come from

- ◆ **Managed services publish directly.** Glue, Lambda and Step Functions need no agent.
- ◆ **The CloudWatch agent** collects logs and metrics from EC2 and on-premises servers.
- ◆ **Vended logs** are service logs AWS delivers for you, such as VPC flow logs.
- ◆ They can go to **CloudWatch Logs, Amazon S3 or Data Firehose**, depending on the service.
- ◆ **Managed syslog ingestion** accepts syslog from network devices directly, since June 2026.

ON THE EXAM

Anything running on an instance you manage needs an agent. Anything managed by AWS publishes on its own.

Vended logs, and where they go

LOG SOURCE	TYPICAL DESTINATION
VPC Flow Logs	CloudWatch Logs or Amazon S3. Network troubleshooting
AWS CloudTrail	A trail to S3, or CloudTrail Lake
S3 server access logs	Another S3 bucket. Object-level request records
Amazon MSK broker logs	CloudWatch Logs, S3 or Data Firehose
Redshift audit logs	CloudWatch Logs or S3. Connection and user activity
Glue and EMR logs	CloudWatch Logs, and S3 for EMR

ON THE EXAM

Destination is often the exam point. S3 for cheap long retention, CloudWatch for interactive investigation.

Three services, three questions. **What is happening, what did someone do, what do the logs say.**

Which service answers which question

QUESTION	SERVICE
Is throughput normal right now?	CloudWatch metrics and alarms
Who deleted that table, and when?	AWS CloudTrail
Why did this job throw an exception?	CloudWatch Logs
Which requests are slow, and where?	AWS X-Ray traces
Has a resource drifted from policy?	AWS Config
What happened across the estate last quarter?	CloudTrail Lake, or logs in S3 queried with Athena

ON THE EXAM

CloudTrail records API calls. It does not record what a job did internally, and it never contains your data.

Metrics, and the alarms on them

- ◆ **Namespaces** group metrics by service. Dimensions identify the specific resource.
- ◆ **Standard metrics** come free with the service. **Custom metrics** you publish yourself.
- ◆ **An alarm** watches one metric against a threshold for a number of periods.
- ◆ **Composite alarms** combine several, to cut noise from correlated failures.
- ◆ **Anomaly detection** builds a band from history, so no fixed threshold is needed.

ON THE EXAM

Anomaly detection is the answer when normal varies. A fixed threshold on a weekly cycle alerts every weekend.

Turning log content into a metric

- ◆ **A metric filter** watches a log group for a pattern and increments a metric.
- ◆ **Count occurrences of a phrase**, such as an error code or a rejection reason.
- ◆ **Extract a numeric value** from a log line and publish it as a metric.
- ◆ **Then alarm on it**, because alarms watch metrics rather than logs.
- ◆ **Standard log class only**. Metric filters are not available on Infrequent Access.

ON THE EXAM

Alarms cannot watch logs directly. A metric filter is the bridge between something written in a log and an alert.

The metrics worth knowing per service

SERVICE	WATCH
AWS Glue	Executor count, memory pressure, shuffle bytes, job run duration
Kinesis Data Streams	Iterator age, write and read throughput exceeded
Amazon DynamoDB	Throttled requests, consumed capacity, successful request latency
Amazon Redshift	Queue wait time, execution time, disk space, concurrency scaling usage
Amazon Athena	Data scanned per query, query queue time, engine execution time
AWS Lambda	Duration, errors, throttles, iterator age for stream sources

ON THE EXAM

Iterator age appears twice. It is the single most diagnostic streaming metric and it means the consumer is falling behind.

Alerting on absence, not just failure

- ◆ A job that **never ran** raises no error and triggers no failure alarm.
- ◆ Treat **missing data as breaching** so an absent metric fires the alarm.
- ◆ Alarm on a **run count** falling below one over the expected window.
- ◆ **EventBridge on a failed state change** catches failures from outside the pipeline.
- ◆ A **notification step inside a workflow** never runs if the workflow died first.

ON THE EXAM

This is module six's point, implemented. The hardest failure to see is the one that produced no event at all.

Getting the notification to a person

- ◆ **Alarm to SNS** is the standard path. Email, SMS, or a chat integration.
- ◆ **SNS fans out**, so one alarm can notify a person and trigger a remediation.
- ◆ **EventBridge rules** react to service events rather than metric thresholds.
- ◆ **Route by severity**. Not everything deserves a phone call at three in the morning.
- ◆ **Alarms nobody acts on** train people to ignore all of them.

ON THE EXAM

An alarm is only useful if somebody does something. The exam does test routing and suppression, not just detection.

A nightly AWS Glue job loads data into Amazon Redshift. An operations team has a CloudWatch alarm on the job's failed run count, notifying an SNS topic. It has worked for a year.

Last week the EventBridge rule that starts the job was disabled during an unrelated change. The job did not run for four nights. No alarm fired, and the problem was found when an analyst noticed stale figures.

What change would have caught this?

- ☐ A Lower the failure alarm's threshold so it fires on a single failure
- ☐ B Alarm on the job's succeeded run count falling below one in twenty four hours, treating missing data as breaching
- ☐ C Add a notification step at the end of the Glue workflow
- ☐ D Enable AWS CloudTrail so the disabling of the rule is recorded

◆ PAUSE AND DECIDE

What change would have caught this?

A Lower the failure threshold

There were no failures. A job that never ran emits no failure metric at any threshold.

C Notification at the end of the workflow

The workflow never started, so its final step never ran. This is the trap from the absence slide.

B Alarm on successes falling below one, missing data breaching

Turns absence into a signal. No run means no success metric, which the alarm now treats as a breach.

D Enable CloudTrail

Records who disabled the rule, after the fact. Useful for the post-mortem and it alerts nobody.

THE SIGNAL

Nothing failed. Every option watching for failure is answering a question the scenario did not ask.

CloudWatch Logs has changed more than almost any service on this exam.

How logs get in, and where they go next

- ◆ **Log groups and streams.** A group per application or service, streams within it.
- ◆ **Retention is per log group**, and defaults to never expiring. That is a cost trap.
- ◆ **Subscription filters** stream matching events onward, in real time.
- ◆ **To Firehose, Kinesis, Lambda or OpenSearch**, for processing or long-term analysis.
- ◆ **Export to Amazon S3** for archival, and for querying with Athena later.

ON THE EXAM

Never expiring is the default retention. On a busy account that is the quiet, permanent line on the bill.

Standard or Infrequent Access log class?

STANDARD

Everything

- Full feature set including metric filters
- Live Tail and subscription filters
- Higher ingestion price per GB
- For logs you actively monitor

INFREQUENT ACCESS

Cheaper to ingest

- Lower ingestion price per GB
- Logs Insights QL, OpenSearch SQL and PPL all supported
- Data protection and export to S3
- For logs you query occasionally

THE TIEBREAKER

The log class is fixed when the group is created. It cannot be changed afterwards.

Searching logs with Logs Insights

- ◆ **Three query languages.** Logs Insights QL, OpenSearch PPL and OpenSearch SQL.
- ◆ **SQL supports joins across log groups** and subqueries, which QL does not.
- ◆ **Field indexes** on up to twenty fields per group speed queries and cut scanned volume.
- ◆ **Scheduled queries** run on a recurring basis, delivering results to S3 or EventBridge.
- ◆ **Live Tail** streams matching events as they arrive, for watching a deployment.

ON THE EXAM

Joining across log groups used to mean exporting to another system. Since December 2024, SQL does it in place.

Masking sensitive data in logs

- ◆ **Applications log more than they should.** Account numbers, emails, tokens.
- ◆ **CloudWatch Logs data protection** detects and masks sensitive patterns automatically.
- ◆ **Masked at ingestion**, so the unmasked value never sits in the log group.
- ◆ **Audited access** can reveal the original, for the people entitled to see it.
- ◆ **Available on Infrequent Access** as well, since March 2026.

ON THE EXAM

Logs are a common place for personal data to leak, and nobody is looking. Detection at ingestion is the answer.

Where to analyse logs, and when

APPROACH	BEST FOR
CloudWatch Logs Insights	Interactive investigation over recent logs, in place
Amazon OpenSearch Service	Dashboards, alerting and search over a large ongoing log estate
Amazon Athena over S3	Occasional analysis of archived logs, at S3 storage prices
Amazon EMR	Very large volumes needing complex processing, not just search
CloudTrail Lake	API activity specifically, with SQL and long retention

ON THE EXAM

The exam guide names all of these. The separator is volume, retention period and whether you need dashboards.

Dashboards and cross-account views

- ◆ **CloudWatch dashboards** combine metrics, alarms and log widgets on one page.
- ◆ **Cross-account observability** lets a monitoring account see many source accounts.
- ◆ **One place to look** matters more than the sophistication of any single graph.
- ◆ **Amazon Managed Grafana** for dashboards spanning AWS and other sources.
- ◆ **Amazon Managed Service for Prometheus** for metrics from container workloads.

ON THE EXAM

Managed Grafana is on the in-scope service list. It appears when dashboards must span more than CloudWatch.

Deeper instrumentation

- ◆ **Container Insights** for ECS and EKS cluster, pod and container metrics.
- ◆ **Lambda Insights** for per-invocation memory, CPU and cold start detail.
- ◆ **Application Signals** tracks service level objectives against real behaviour.
- ◆ **Embedded metric format** publishes metrics inside log events, avoiding a separate API call.
- ◆ Each adds cost. Instrument what you will act on.

ON THE EXAM

Embedded metric format is the efficient way for a high-volume function to publish custom metrics.

A company must retain application logs for seven years to satisfy a regulator, and expects to query them perhaps twice a year during audits. Logs currently go to CloudWatch Logs with no retention policy set, and the CloudWatch bill has grown steadily.

The security team also needs to run occasional forensic searches over the last thirty days, interactively.

Which approach best meets both needs?

- A** Keep everything in CloudWatch Logs indefinitely and query with Logs Insights
- B** Set a thirty day retention on the log groups, and export older logs to Amazon S3 with a lifecycle policy to Glacier, queried with Athena
- C** Stream all logs to Amazon OpenSearch Service and retain seven years of indices
- D** Reduce retention to thirty days and accept the loss of older logs

◆ PAUSE AND DECIDE

Which approach best meets both needs?

- A** Everything in CloudWatch forever
Meets both requirements at CloudWatch Logs storage prices for seven years. That is the bill they complained about.
- B** Thirty day retention, export to S3, lifecycle to Glacier, query with Athena
Recent logs stay interactive; archives move to storage priced for the access pattern and stay queryable.
- C** Seven years of OpenSearch indices
Keeps seven years of data on a cluster sized for search, for two queries a year. The most expensive option here.
- D** Thirty days only
Solves the cost problem by breaking the regulatory requirement. Never the answer.

THE SIGNAL

Two access patterns, thirty days and seven years, need two storage tiers. This is module eight's lifecycle argument applied to logs.

CloudTrail answers **who**, and it is not a debugging tool.

What CloudTrail records, and what it misses

- ◆ **Management events** by default. Creating, deleting, configuring resources.
- ◆ **Data events are off by default**, and charged. S3 object access, Lambda invocations.
- ◆ **A trail to Amazon S3** gives durable, long-term storage of the record.
- ◆ **CloudTrail Lake** stores events for years and queries them with SQL directly.
- ◆ **CloudTrail Insights** flags unusual API call rates automatically.

ON THE EXAM

Data events are the ones people expect and do not have. Reading an S3 object is not recorded unless you enabled it.

Trail to S3, or CloudTrail Lake?

TRAIL TO AMAZON S3

Files you own

- JSON files delivered to a bucket
- Query with Athena after defining a table
- Cheap storage, lifecycle rules apply
- You build the querying yourself

CLOUDTRAIL LAKE

A managed event store

- SQL queries with no setup
- Retention up to ten years
- Can include events from outside AWS
- Priced for ingestion and querying

THE TIEBREAKER

Lake is the lower-overhead answer for audit questions. A trail to S3 is cheaper if you already query logs with Athena.

Tracing a request across services

- ◆ **AWS X-Ray** follows a single request through several services.
- ◆ A **service map** shows where the time went, and which call failed.
- ◆ **Correlation identifiers** do the same job in logs, if you log them consistently.
- ◆ **Without either**, connecting a symptom to a cause across five services is guesswork.
- ◆ Useful for API-driven pipelines, less so for long batch jobs.

ON THE EXAM

X-Ray suits request-response work. A four hour Glue job is better served by Spark UI and job metrics.

Every pipeline failure this course has described falls into **four families**.

The four families, and their signatures

FAMILY	HOW IT PRESENTS
Not enough capacity	All workers busy, everything uniformly slow, scales with volume
Uneven distribution	One task or slice at full load while the rest idle
Waiting	Low resource usage, nothing progressing, no errors
Silently wrong	Everything succeeds, and a number is incorrect

ON THE EXAM

Adding capacity fixes the first family and nothing else. The exam offers it for all four.

Troubleshooting a Glue job

- ◆ **Continuous logging** streams driver and executor logs during the run, not after.
- ◆ **The Spark UI** shows stages and tasks, and where time was actually spent.
- ◆ **Job run insights** names the failing line and suggests a root cause.
- ◆ **CloudWatch metrics** for executor count, memory pressure and shuffle bytes.
- ◆ **A four hour job that fails at hour three** gives no logs at all without continuous logging.

ON THE EXAM

Module two covered these as configuration. Here they are the diagnostic toolkit, and the exam names Glue explicitly.

Troubleshooting EMR and MWAA

- ◆ **EMR writes logs to S3** when configured, which survives cluster termination.
- ◆ **A transient cluster that terminated** takes its local logs with it.
- ◆ **YARN and Spark history servers** for application-level diagnosis.
- ◆ **MWAA writes scheduler, worker, web server and task logs** to CloudWatch Logs.
- ◆ **MWAA log levels are configurable per component**, which matters for cost and noise.

ON THE EXAM

Configuring EMR log delivery to S3 before you need it is the difference between diagnosing a failure and guessing.

Troubleshooting a warehouse or a stream

- ◆ **Redshift query history** separates queue time from execution time, as module ten covered.
- ◆ **SYS views** for locks, disk usage and query plans on current generations.
- ◆ **Kinesis iterator age** rising means consumers cannot keep up with producers.
- ◆ **Rising iterator age plus retention** is a countdown to data loss, not just latency.
- ◆ **Throttling metrics** distinguish a hot key from genuine capacity shortage.

ON THE EXAM

An iterator age approaching the retention window means records will be lost, not merely delayed. That is urgent.

An Amazon EMR transient cluster runs a nightly Spark job and terminates on completion. Last night the job failed after two hours. The cluster terminated automatically and the operations team can see only that the step failed.

There are no application logs available. The team needs to diagnose this failure and prevent the same blind spot recurring.

What should the data engineer do?

- ☐ A Rerun the job on a long-running cluster and reproduce the failure to read the logs
- ☐ B Configure the cluster to archive logs to Amazon S3, and enable the persistent Spark history server
- ☐ C Increase the cluster size so the job does not fail
- ☐ D Enable AWS CloudTrail to capture what the job did

◆ PAUSE AND DECIDE

What should the data engineer do?

- A

 Reproduce on a long-running cluster
May work for this one failure, at the cost of a rerun and a cluster. It fixes nothing for next time.
- B

 Archive logs to S3, enable the persistent history server
Logs survive termination, and the history server keeps Spark's own view of the run. Diagnosis becomes possible for every future failure.
- C

 A bigger cluster
Capacity for a failure nobody has diagnosed. It might be skew, bad data or a code error.
- D

 Enable CloudTrail
Records the API calls that created the cluster and submitted the step. It contains nothing about why Spark failed.

THE SIGNAL

Option D is the CloudTrail confusion, tested directly. API activity is not application behaviour.

Diagnosing across the pipeline

- ◆ **Start at the symptom and work backwards**, not forwards from the source.
- ◆ **Check the schedule fired**, then that the job ran, then that it read what it expected.
- ◆ **Compare row counts in and out**. A drop between stages localises the problem.
- ◆ **Check whether upstream data changed**, which module twelve covered.
- ◆ **A pipeline that succeeded** still needs checking when the output is wrong.

ON THE EXAM

Rows in and rows out, emitted as metrics per stage, turns a hunt through five services into reading one graph.

Monitoring the data, not just the pipeline

- ◆ **Row counts per stage** catch data disappearing between steps.
- ◆ **Null rates per column** catch a source that started sending blanks.
- ◆ **Distribution checks** catch a value range shifting unexpectedly.
- ◆ **Freshness checks** catch data that stopped arriving while the job kept succeeding.
- ◆ **None of these are infrastructure metrics.** They come from the data itself.

ON THE EXAM

Every metric elsewhere in this module describes the pipeline. These describe what came out of it.

Extracting logs for an audit

- ◆ **CloudTrail for API activity**, to a trail in S3 or into CloudTrail Lake.
- ◆ **Export CloudWatch Logs to S3** for application evidence over long periods.
- ◆ **Query with Athena** once exported, defining a table over the log format.
- ◆ **Object Lock on the archive bucket** so evidence cannot be altered, as module eight covered.
- ◆ **Consistent retention across sources**, or the audit trail has holes.

ON THE EXAM

An auditor wants immutability as much as completeness. Compliance mode Object Lock is what provides it.

Watching the limits themselves

- ◆ **Service Quotas** shows current limits and usage, and requests increases.
- ◆ **Alarm on approaching a quota**, rather than discovering it during a peak.
- ◆ **Some quotas are hard** and no request raises them. Lambda's fifteen minutes, for instance.
- ◆ **Throttling is often a quota**, not a capacity problem in the usual sense.
- ◆ **AWS Health** notifies about events affecting your account and resources.

ON THE EXAM

A pipeline growing steadily will meet a quota eventually. Alarming at eighty percent turns a failure into a ticket.

When the problem is not yours

- ◆ **AWS Health** reports events affecting the services and resources you use.
- ◆ **Personalised**, so it names your affected resources rather than a general status page.
- ◆ **EventBridge can react to Health events**, so degradation reaches your alerting.
- ◆ **Scheduled changes too**, such as a certificate rotation or a required upgrade.
- ◆ Investigating your own pipeline for hours during a service event is avoidable.

ON THE EXAM

Routing Health events into the same alerting as your own alarms stops a service issue looking like a code issue.

Responding without a human

- ◆ An alarm can trigger an action, not only a notification.
- ◆ Systems Manager Automation runbooks perform a defined remediation.
- ◆ EventBridge to Lambda for a bespoke response to a specific failure.
- ◆ Automate only what you understand. Automatic retries on an unknown failure hide it.
- ◆ Record what the automation did, or the next person cannot explain the system.

ON THE EXAM

Automating a known, understood remediation is good. Automating a mystery makes the mystery permanent.

Writing down what to do

- ◆ **A runbook** records what an alarm means and what to check first.
- ◆ **Link it from the alarm description**, so it is found at three in the morning.
- ◆ **Record the last few incidents** and what actually resolved them.
- ◆ **Systems Manager documents** hold the procedure, and can execute it.
- ◆ **Knowledge in one engineer's head** is an availability risk, not a monitoring one.

ON THE EXAM

The exam rarely asks about runbooks directly. It regularly asks about reducing time to resolution, which is the same thing.

Testing that monitoring works

- ◆ **An untested alarm** is an assumption, not a control.
- ◆ **Trigger it deliberately** and confirm somebody actually receives the notification.
- ◆ **Check the destination too.** An SNS subscription that was never confirmed sends nothing.
- ◆ **Review alarms that never fire**, because they may be watching the wrong thing.
- ◆ **Review alarms that always fire**, because people have learned to ignore them.

ON THE EXAM

The first scenario in this module is an alarm that worked correctly for a year and could never have caught the failure.

What monitoring itself costs

- ◆ **Log ingestion** is the largest charge, per gigabyte, and grows with verbosity.
- ◆ **Storage forever** is the default. Retention policies are the main lever.
- ◆ **Logs Insights charges per gigabyte scanned**, so field indexes reduce cost as well as time.
- ◆ **Custom metrics are charged per metric**, and high-cardinality dimensions multiply them.
- ◆ **Infrequent Access log class** cuts ingestion price for logs you rarely read.

ON THE EXAM

A metric with a dimension per customer becomes a metric per customer. That is how a monitoring bill explodes.

A data platform team runs forty Glue jobs and twelve Step Functions workflows. When a business report is wrong, diagnosing which stage caused it takes most of a day, because each investigation means opening logs for several services and correlating timestamps by hand.

The team wants to reduce that investigation time. Jobs generally succeed, so failure alerting is not the gap.

What should the team implement?

- ☐ A Increase the CloudWatch Logs retention period so more history is available
- ☐ B Emit per-stage row counts as custom metrics and log a correlation identifier through every component
- ☐ C Enable AWS X-Ray tracing on the Glue jobs
- ☐ D Add failure alarms to every Glue job and Step Functions state machine

◆ PAUSE AND DECIDE

What should the team implement?

- A

 Longer retention
Keeps more of the logs that already take a day to correlate. It makes the same investigation possible for longer.
- B

 Per-stage counts plus a correlation identifier
Counts localise which stage lost data; the identifier joins one run's records across every service. Both halves of the problem.
- C

 X-Ray on Glue jobs
Traces request-response paths. A batch job running for an hour is the wrong shape for it.
- D

 Failure alarms everywhere
The question states jobs generally succeed. More failure alerting addresses a problem they do not have.

THE SIGNAL

"Jobs generally succeed" rules out every failure-detection option. The problem is diagnosis, not detection.

Symptoms and their causes

SYMPTOM	LIKELY CAUSE
No alert, and nothing ran	Alarming on failure only. Alarm on absence instead
Alert storm from one incident	Correlated alarms. Use a composite alarm
Alerts every weekend, none on weekdays	A fixed threshold on a cyclical metric. Use anomaly detection
No logs from a failed long job	Continuous logging not enabled, or a terminated transient cluster
Cannot find who read an object	S3 data events are off by default in CloudTrail
CloudWatch bill growing steadily	Default retention of never expire, or high-cardinality custom metrics

ON THE EXAM

Three of these six are configuration defaults that nobody changed. Defaults are where monitoring gaps live.

Reading a monitoring question quickly

- ◆ **Which of the three questions?** What is happening, who did it, or what do logs say.
- ◆ **Did something fail, or did nothing happen?** Absence needs a different alarm.
- ◆ **Which failure family?** Capacity, distribution, waiting, or silently wrong.
- ◆ **How long must logs be kept, and how often read?** That picks the storage tier.
- ◆ **Is a default the cause?** Retention, data events and missing-data handling all default unhelpfully.

ON THE EXAM

Next: data quality, skew and sampling. Then cost and performance closes Domain 3.